

JSON

peanutixx

2026 年 5 月 14 日

Contents

1 JSON 的数据结构	1
1.1 基本数据类型	1
1.2 复合数据类型	2
1.3 嵌套结构	2
2 安装 nlohmann/json	2
3 使用 nlohmann/json	3
3.1 序列化和反序列化	3
3.2 构建 nlohmann/json 对象	3
3.3 解析 nlohmann/json 对象	5
4 练习题	7
4.1 将 Student 对象转换成 JSON	7
4.2 将 JSON 字符串解析为 C++ 对象	8
4.3 解析 JSON 数组	8
4.4 XML 转 JSON*	8
4.5 JSON 转 XML*	11

JSON (JavaScript Object Notation) 是一种轻量级的数据交换格式¹ 它易于人类阅读和编写，同时也易于机器解析和生成。JSON 是基于 JavaScript 的一个子集，但它独立于编程语言。几乎所有的现代编程语言都支持 JSON。

JSON 的重要性

JSON 已经成为现代 Web 开发中事实上的数据交换标准，掌握它对于任何开发者来说都是必不可少的技能。甚至，许多主流的编程语言都已经内置（原生支持）JSON 的解析和处理能力，无需安装任何第三方库。比如：JavaScript、Python、Go、C# 等。

1 JSON 的数据结构

1.1 基本数据类型

JSON 支持以下几种基本数据结构，但所有复杂数据结构均由这些基本数据结构组合而成。

¹ 数据交换格式是一种在不同平台、不同编程语言之间，用于结构化传输和存储数据的标准化格式。这种格式必须是平台无关和语言无关的。常见的数据交换格式有：XML、JSON、Protobuf 等。

类型	示例	说明
字符串	"花生"	字符串必须用""括起来
数字	18, 3.14	数字可以是整数，也可以是浮点数
布尔值	true, false	true 或 false（没有双引号）
null	null	表示空值

1.2 复合数据类型

JSON 还支持两种复合数据类型：对象（*Object*）和数组（*Array*）。

类型	示例	说明
对象	{"name": "花生", "age": 18}	对象是键值对的集合，用{}包裹，键必须是字符串
数组	["apple", "banana", 123]	数组表示有序列表，用[]包裹，元素可以是混合类型

1.3 嵌套结构

JSON 允许多层嵌套，从而构建复杂数据。如：

```
{
  "user": {
    "name": "花生",
    "age": 18,
    "contacts": [
      {
        "type": "email",
        "value": "peanut@example.com"
      },
      {
        "type": "phone",
        "value": "1234567890"
      }
    ],
    "is_active": true
  }
}
```

JSON 的数据结构虽然简单（仅 6 种类型），但通过对象和数组的嵌套，可以描述绝大多数复杂数据，这也是它成为现代数据交换标准的原因。

2 安装 nlohmann/json

[nlohmann/json](#) 是一个 header-only 的库，所以，我们只要将它的头文件移动到 `/usr/local/include` 目录下即可：

Terminal

```
$ sudo mv nlohmann /usr/local/include
```

3 使用 nlohmann/json

数据交换格式的处理流程都大同小异，以 JSON 为例，它的处理流程如下：

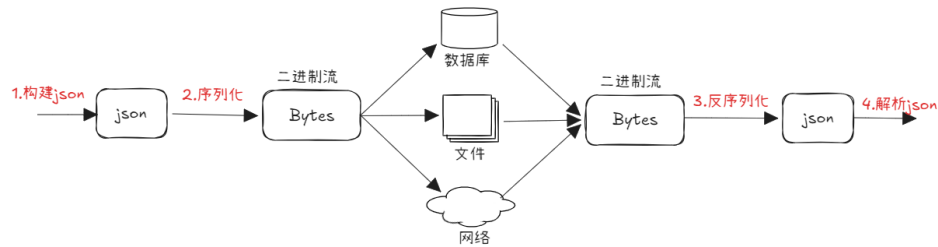


Figure 1: JSON 处理流程

数据交换格式有两个核心的操作：

- 序列化：将内存中的数据结构或对象转换为可存储或可传输的格式（满足数据交换格式约束的字节流）的过程。
- 反序列化：是序列化的逆过程，将字节流恢复为内存中的数据结构或对象。

3.1 序列化和反序列化

```
01_json_basics.cc
1 #include <fstream>
2 #include <iostream>
3 #include <nlohmann/json.hpp>
4
5 using namespace std;
6 using json = nlohmann::json;
7
8 int main()
9 {
10     // 1. 解析文件
11     ifstream f("example.json");
12     json j1 = json::parse(f);
13     // cout << j.dump() << endl;
14     cout << j1.dump(2) << endl; // 以漂亮格式输出，缩进单位为 2 个空格
15     // 2. 解析字符串（裸字符串）
16     json j2 = json::parse(R"({
17         "pi": 3.141,
18         "happy": true
19     })");
20     cout << j2.dump(2) << endl;
21 }
```

在 nlohmann/json 中，序列化操作为 `data.dump()`，反序列化操作为 `json::parse(...)`，使用起来非常简单 (so easy...)。接下来，我们重点学习下：如何构建 nlohmann/json 对象和如何解析 nlohmann/json 对象。

3.2 构建 nlohmann/json 对象

```
02_build_json.cc
1 #include <fstream>
```

```

2 #include <iostream>
3 #include <list>
4 #include <nlohmann/json.hpp>
5
6 using namespace std;
7 using json = nlohmann::json;
8
9 int main()
10 {
11     // 1. 基本数据结构: 直接赋值
12     json j1;
13     cout << j1.dump() << endl; // null
14     json j2 = 3.14159;
15     cout << j2.dump() << endl; // 3.14159
16     json j3 = false;
17     cout << j3.dump() << endl; // false
18     json j4 = "花生";
19     cout << j4.dump() << endl; // "花生"
20     // 2. nlohmann/json 支持初始化列表{...}
21     json j5 = { "peanut", "loves", "xixi", 520 }; // 数组
22     cout << j5.dump() << endl; // ["peanut","loves","xixi",520]
23     json j6 = { // 对象
24         { "happy", true },
25         { "pi", 3.141 }
26     };
27     cout << j6.dump() << endl; // {"happy":true,"pi":3.141}
28     // 3. 解决歧义
29     // Q1: 如何表示空数组和空对象?
30     json j7 = {};
31     cout << j7.dump() << endl; // null
32     json j9 = json::array();
33     cout << j9.dump() << endl; // []
34     json j10 = json::object();
35     cout << j10.dump() << endl; // {}
36     // Q2: 如何表示: [{"currency", "USD"}, {"value", 42.99}]
37     json d1 = { { "currency", "USD" }, { "value", 42.99 } };
38     cout << d1.dump() << endl; // { "currency": "USD", "value": 42.99 }
39     // 遇到歧义的时候, 我们需要显示指明是数组还是对象
40     json d2 = json::array({ { "currency", "USD" }, { "value", 42.99 } });
41     cout << d2.dump() << endl; // [{"currency", "USD"}, {"value", 42.99}]
42     json d3 = json::object({ { "currency", "USD" }, { "value", 42.99 } });
43     cout << d3.dump() << endl; // {"currency": "USD", "value": 42.99}
44     // 4. 动态构建 (适用于 json 中的数据来自多个不同数据结构的场景)
45     json d4;
46     d4["pi"] = 3.141;
47     d4["happy"] = true;
48     d4["name"] = "Niels";
49     d4["nothing"] = nullptr;
50     d4["answer"]["everything"] = 42;
51     d4["list"] = { 1, 0, 2 };
52     d4["object"] = { { "currency", "USD" }, { "value", 42.99 } };

```

```

53     cout << d4.dump(2) << endl;
54     json d5;
55     d5["pi"] = 3.141;
56     d5["happy"] = true;
57     d5["name"] = "Niels";
58     d5["nothing"] = nullptr;
59     d5["answer"]["everything"] = 42;
60     // 向数组中添加元素
61     d5["list"].push_back(1);
62     d5["list"].push_back(0);
63     d5["list"].push_back(2);
64     // 向对象中添加键值对
65     d5["object"].emplace("currency", "USD");
66     d5["object"].emplace("value", 42.9);
67     cout << d5.dump(2) << endl;
68     // 5. STL 容器可以轻松地转换成 JSON 的数组或对象
69     // 数组 <-- array, vector, deque, list, forward_list, stack, queue
70     // 数组 <-- set, multiset, unordered_set, unordered_multiset
71     list<double> numbers = { 3.141, 2.728, 1.414, 1.681, 0.577 };
72     json d6 = numbers;
73     cout << d6.dump() << endl; // [3.141, 2.728, 1.414, 1.681, 0.577]
74     // 对象 <-- map, unordered_map, multimap, unordered_multimap
75     map<string, string> couples = {
76         { "刘强东", "章泽天" },
77         { "文章", "马伊琍" },
78         { "李小璐", "贾乃亮" },
79         { "马蓉", "王宝强" }
80     };
81     json d7 = couples;
82     cout << d7.dump() << endl;
83 }

```

从上面的示例程序，我们可以看到构建 `nlohmann/json` 对象是很容易的：

- 对于 JSON 基本数据结构，我们直接赋值即可。
- `nlohmann/json` 支持初始化列表。
- 遇到歧义时，我们可以通过 `json::array()` 和 `json::object()` 指定到底要创建数组还是对象。
- 使用 `push_back(element)` 往数组中追加元素，使用 `emplace(key, val)` 往对象中添加元素。
- STL 容器可以很轻松地转换成 JSON 中的数组或对象。

3.3 解析 `nlohmann/json` 对象

```

03_parse_json.cc
1 #include <ios>
2 #include <iostream>
3 #include <nlohmann/json.hpp>
4
5 using namespace std;
6 using json = nlohmann::json;
7

```

```

8 int main()
9 {
10     // 1. 判断 JSON 的数据类型
11     json data;
12     data.is_null();
13     data.is_number();
14     data.is_boolean();
15     data.is_string();
16     data.is_array();
17     data.is_object();
18     // 2. 解析基本数据类型: 直接赋值
19     json j1 = 3.14; // number
20     double pi = j1;
21     cout << pi << endl; // 3.14
22     json j2 = true; // boolean
23     bool flag = j2;
24     cout << boolalpha << flag << endl; // true
25     json j3 = "茜茜";
26     string name = j3;
27     cout << name << endl; // "茜茜"
28     // 3. nlohmann/json 拥有 getter 方法, 可以显示指定转换后的类型
29     json d1 = 3.14; // number
30     double pie = d1.get<double>();
31     cout << pie << endl; // 3.14
32     json d2 = true; // boolean
33     bool ok = d2.get<bool>();
34     cout << boolalpha << ok << endl; // true
35     json d3 = "茜茜";
36     string idol = d3.get<string>();
37     cout << idol << endl; // "茜茜"
38     // 4. 解析数组
39     json array = { "peanut", "loves", "xixi", 520 };
40     // 访问元素
41     cout << array[0] << " "
42         << array[1] << " "
43         << array[2] << " "
44         << array[3] << endl; // "peanut" "loves" "xixi" 520
45     // 遍历数组
46     for (auto& element : array) {
47         cout << element << " ";
48     }
49     cout << endl;
50     // 5. 解析对象
51     json object = {
52         { "id", 1 },
53         { "name", "花生" },
54         { "age", 18 }
55     };
56     // 根据键访问值
57     cout << object["id"] << " "
58         << object["name"] << " "

```

```

59         << object["age"] << endl; // 1 "花生" 18
60     // 遍历对象
61     for (auto& entry : object.items()) {
62         cout << entry.key() << ": " << entry.value() << endl;
63     }
64 }

```

从上面的示例程序，我们可以看到解析 `nlohmann/json` 对象也是很容易的：

- `nlohmann/json` 有 `is_xxx()` 方法，可以判断 JSON 的具体数据类型。
- 对于基本数据类型，直接赋值就可以完成解析。
- 当然我们也可以使用 `getter` 方法，显示指定转换后的类型。
- 我们可以通过取下标的方式访问数组的元素，也可以通过基于范围的 `for` 循环遍历数组。使用方式和 `std::vector` 是一致的。
- 我们可以通过取下标的方式来访问键对应的值，也可以通过基于范围的 `for` 循环遍历对象。使用方式和 `std::map` 略有区别，需要格外注意。

熟练掌握上面的内容，足以应付工作中 90% 以上的 JSON 相关问题。更多详细信息，请查阅 [JSON for Modern C++](#)。

4 练习题

4.1 将 Student 对象转换成 JSON

```

struct Student {
    int id;
    string name;
    int chinese;
    int math;
    int english;
    vector<string> hobbies;
    bool isGraduated;
};

Student s = { 1001, "花生", 92, 95, 88, { "唱歌", "跳舞", "rap", "篮球" }, false };

```

将上面的学生对象 `s`，转换成下面的 JSON 对象（字段的顺序可以不一样）：

```

{
    "id": 1001,
    "name": "花生",
    "scores": {
        "chinese": 92,
        "english": 88,
        "math": 95
    },
    "hobbies": ["唱歌", "跳舞", "rap", "篮球"],
    "isGraduated": false
}

```

参考答案：见 `examples/exercise01.cc`。

4.2 将 JSON 字符串解析为 C++ 对象

将以下 JSON 字符串解析为 C++ 对象，并获取其中的书名、价格和标签：

```
{
  "book": {
    "title": "JavaScript 高级程序设计",
    "author": "Nicholas C. Zakas",
    "price": 129.00,
    "publisher": "人民邮电出版社",
    "tags": ["前端", "JavaScript", "编程"]
  }
}
```

参考答案：见 `examples/exercise02.cc`。

4.3 解析 JSON 数组

给定 JSON 数组，回答下面的问题：

```
[
  {
    "name": "赵一",
    "age": 18,
    "gender": "男",
    "scores": [85, 92, 78]
  },
  {
    "name": "钱二",
    "age": 19,
    "gender": "女",
    "scores": [96, 88, 94]
  },
  {
    "name": "孙三",
    "age": 18,
    "gender": "男",
    "scores": [76, 81, 69]
  }
]
```

1. 计算每个学生的平均分。
2. 找出所有 18 岁的学生，并打印他们的名字。
3. 找出总分最高的学生，并打印他们的名字。

参考答案：见 `examples/exercise03.cc`。

4.4 XML 转 JSON*

1. 将下面的 XML 转换为 JSON，并输出（各种数据交换格式之间是可以相互转换的）：

```
<library>
  <book id="B001">
    <title>三体</title>
    <author>刘慈欣</author>
    <year>2008</year>
    <price currency="CNY">68.00</price>
  </book>
  <book id="B002">
    <title>百年孤独</title>
    <author>加西亚·马尔克斯</author>
    <year>2011</year>
    <price currency="CNY">55.00</price>
  </book>
</library>
```

解析上面的 XML 数据，得到下面的结果：

```
[
  {
    "@id": "B001",
    "author": "刘慈欣",
    "price": {
      "@currency": "CNY",
      "value": 68.0
    },
    "title": "三体",
    "year": 2008
  },
  {
    "@id": "B002",
    "author": "加西亚·马尔克斯",
    "price": {
      "@currency": "CNY",
      "value": 55.0
    },
    "title": "百年孤独",
    "year": 2011
  }
]
```

参考答案：

```
exercise04.cc
1 #include <iostream>
2 #include <nlohmann/json.hpp>
3 #include <tinyxml2.h>
4
5 using namespace std;
6 using namespace tinyxml2;
7 using json = nlohmann::json;
8
```

```

9  const char* xmlstring = R"(
10 <library>
11     <book id="B001">
12         <title>三体</title>
13         <author>刘慈欣</author>
14         <year>2008</year>
15         <price currency="CNY">68.00</price>
16     </book>
17     <book id="B002">
18         <title>百年孤独</title>
19         <author>加西亚·马尔克斯</author>
20         <year>2011</year>
21         <price currency="CNY">55.00</price>
22     </book>
23 </library>
24 )";
25
26 int main()
27 {
28     // 1. 解析 XML 字符串
29     XmlDocument doc;
30     if (doc.Parse(xmlstring) != XML_SUCCESS) {
31         cerr << "Failed to parse XML string" << endl;
32         return -1;
33     }
34     // 2. 获取 library 根元素
35     XElement* library = doc.FirstChildElement();
36     if (!library) {
37         cerr << "No library element found" << endl;
38         return -1;
39     }
40     // 3. 解析所有 book 元素，添加到 result 数组
41     json result = json::array();
42     XElement* book = library->FirstChildElement("book");
43     while (book) {
44         json bookJson;
45         // 获取属性 id
46         const char* id = book->Attribute("id");
47         if (id) {
48             bookJson["@id"] = id; // 使用@前缀表示属性
49         }
50         // 获取子元素 title
51         XElement* title = book->FirstChildElement("title");
52         if (title) {
53             bookJson["title"] = title->GetText();
54         }
55         // 获取子元素 author
56         XElement* author = book->FirstChildElement("author");
57         if (author) {
58             bookJson["author"] = author->GetText();
59         }

```

```

60     // 获取子元素 year
61     XElement* year = book->FirstChildElement("year");
62     if (year) {
63         bookJson["year"] = stoi(year->GetText());
64     }
65     // 获取子元素 price 及其属性
66     XElement* price = book->FirstChildElement("price");
67     if (price) {
68         json priceJson;
69         // 获取 currency 属性
70         const char* currency = price->Attribute("currency");
71         if (currency) {
72             priceJson["@currency"] = currency;
73         }
74         // 获取 price 的文本内容
75         const char* text = price->GetText();
76         if (text) {
77             priceJson["value"] = stod(text);
78         }
79         bookJson["price"] = priceJson;
80     }
81     // 将 bookJson 追加到 result 数组
82     result.push_back(bookJson);
83     // 获取下一个 book 元素
84     book = book->NextSiblingElement("book");
85 }
86 cout << result.dump(2) << endl;
87 }

```
